VIT CHENNAI

EMBEDDED SYSTEM

PROF Dr. Sritama Roy

DONE BY
23BCE5104 LOKESWARAN S
23BCE5103 DHANUSH K

TITLE :

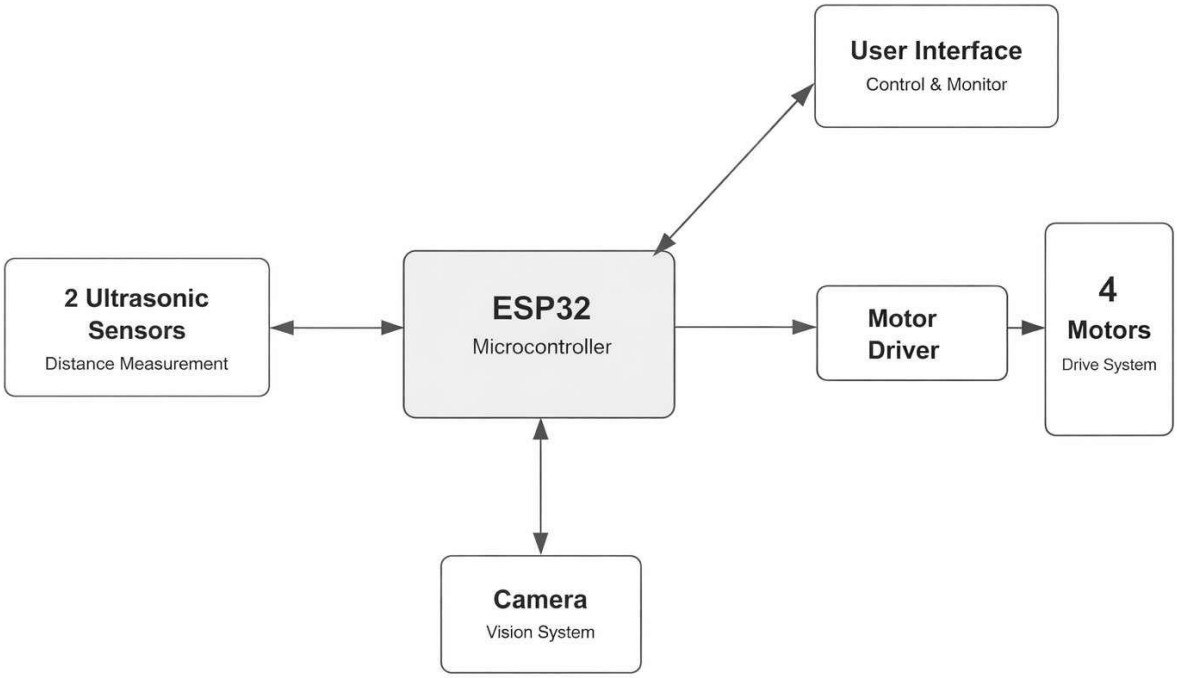
Drive Tutor: A Real-Time Automated Driver Assistance and Training System

OBJECTIVE:

The primary objective of this project is to design and develop an advanced web-controlled RC car system that integrates real-time FPV (First-Person View) streaming, an interactive driving dashboard, and intelligent safety mechanisms. The system aims to:

1. Provide a realistic driving experience through a browser-based interface with simulated physics (RPM, speed, gears, fuel).
2. Enable remote vehicle control using microcontrollers (ESP8266 and ESP32-CAM) over network communication.
3. Enhance driving quality using an AI-powered Driving Instructor that analyzes telemetry and provides corrective feedback.
4. Improve safety by implementing an Autonomous Emergency Braking (AEB) system using real-time ultrasonic sensor data.
5. Demonstrate seamless integration of hardware, software, and AI systems in a unified control architecture.

BLOCK DIAGRAM:



COMPONENTS/ SOFTWARE REQUIRED:

S. NO.	HARDWARE/SOFTWARE	COMPONENT
1	Hardware	Node MCU
2	Hardware	HC-SR04 (Ultra sonic sensor x 2)
3	Hardware	L293D-IC (Instead of motor driver)
4	Hardware	ESP32-CAM
5	Hardware	DC motor *2
6	Hardware	Switch
7	Hardware	Buck Converter
8	Hardware	Battery Pack: Lithium-ION battery (2000mAh x 3)
9	Hardware	Chassis + 2 Wheels +Ball caster wheel
10	Software	Arduino IDE
11	Software	VS Code

PROJECT DESCRIPTION:

This project is an advanced, web-based Remote Control (RC) car system featuring a First-Person View (FPV) video stream, an immersive interactive dashboard, and intelligent safety features. The system pairs a high-performance browser interface—complete with functional pedals, a responsive steering wheel, and physics-based engine simulation—with microcontrollers (ESP32-CAM and ESP8266) to navigate the physical vehicle. It is heavily augmented by a custom AI Driving Instructor powered by the Gemini API, which monitors telemetry for driving mistakes. Additionally, it implements an Autonomous Emergency Braking (AEB) system that reacts instantly to physical ultrasonic sensor data to prevent collisions.

CONCEPT LEARNED:

Microcontroller Networking: Managing asynchronous HTTP communication across dual microcontrollers (ESP32-CAM for FPV streaming and an ESP8266 logic server for motor control and sensor polling).
Physics & Engine Simulation: Using JavaScript to synthesize real-time behavior for RPM, speed, and fuel consumption based on continuous inputs (throttle, brake, clutch) and active gear states.

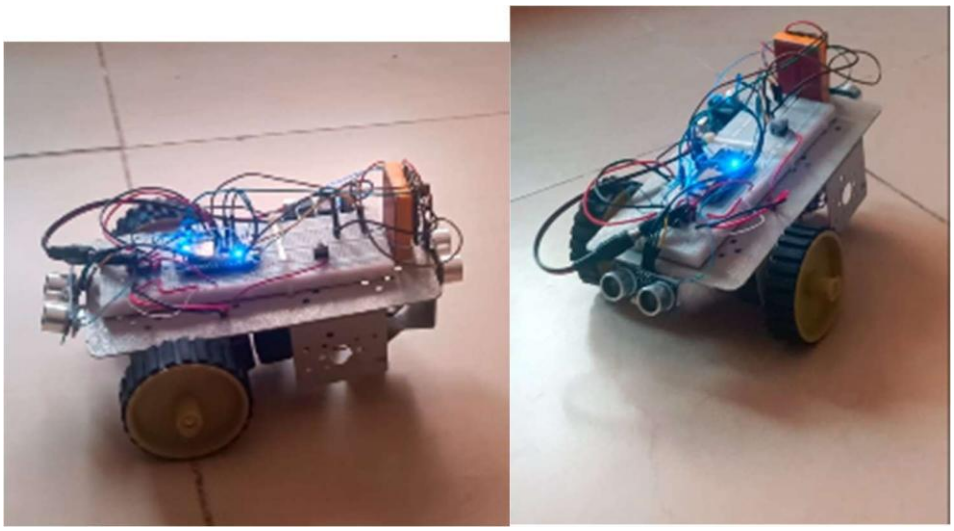
AI API Integration: Constructing complex prompts systematically and sending telemetry data to the Gemini API to produce localized, conversational, and corrective user feedback on exactly how the user was driving.
Complex Event Handling: Orchestrating simultaneous DOM events across touch screens, mouse dragging, and keyboard listeners to create smooth, non-blocking UI controls (e.g., turning a steering wheel while pressing the brake).
Autonomous Safety Systems: Implementing software-in-the-loop logic like AEB, which overrides user throttle input to automatically halt the car exclusively when physical proximity sensors detect an imminent collision.

IMPLEMENTATION:

Hardware Layer: The ESP8266 functions as the main logical receiver, processing HTTP parameters for motor and auxiliary functions, while the ESP32-CAM operates its own server to deliver the FPV video feed. Ultrasonic sensors supply front and rear distance data continuously.

Frontend Interface: Built with clean HTML, styled with CSS for custom gauges/pedals, and driven by vanilla JavaScript. It relies on intermittent data dispatching loops with anti-flooding logic to guarantee high responsiveness without overwhelming the microcontrollers.

AI Instructor Engine: Tracks driving infractions (like failing to indicate during sharp turns, oversteering at high speeds, or skipping gears maliciously) inside an array loop. This array acts as the driving payload, translating raw numerical errors into plain English phrases that Gemini parses back into helpful guidance via a modal.

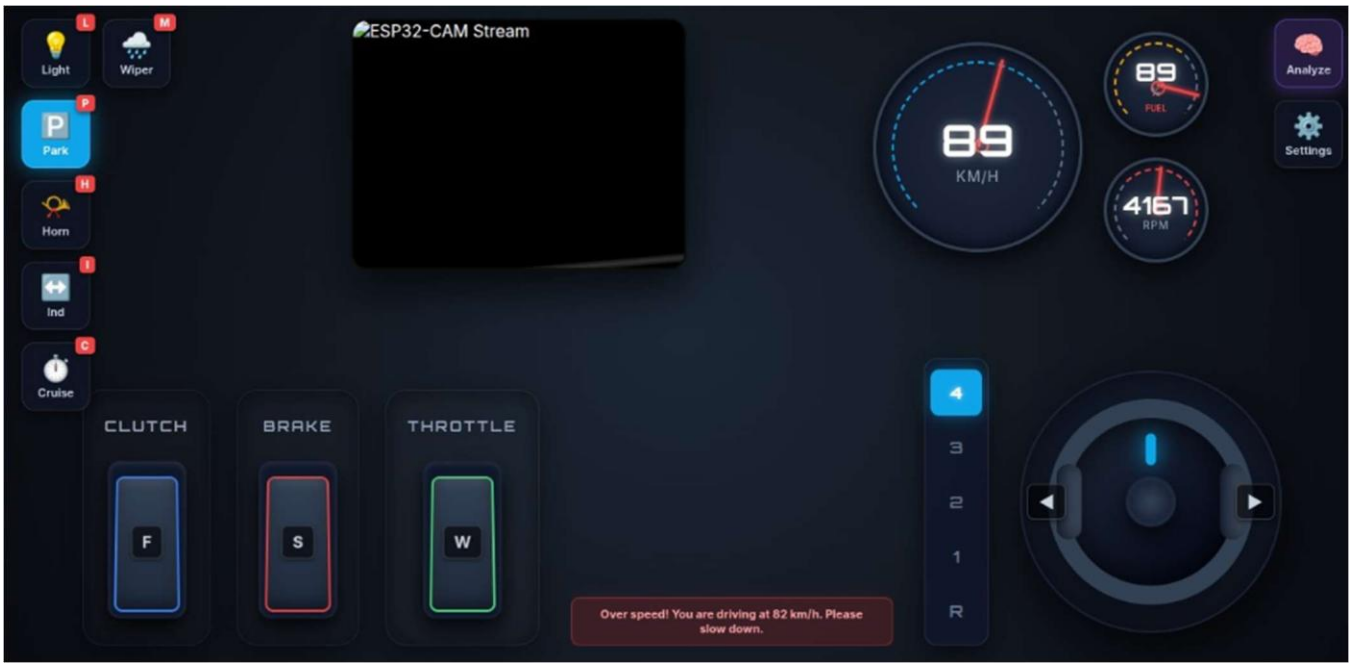


Drive tutor(Prototype)

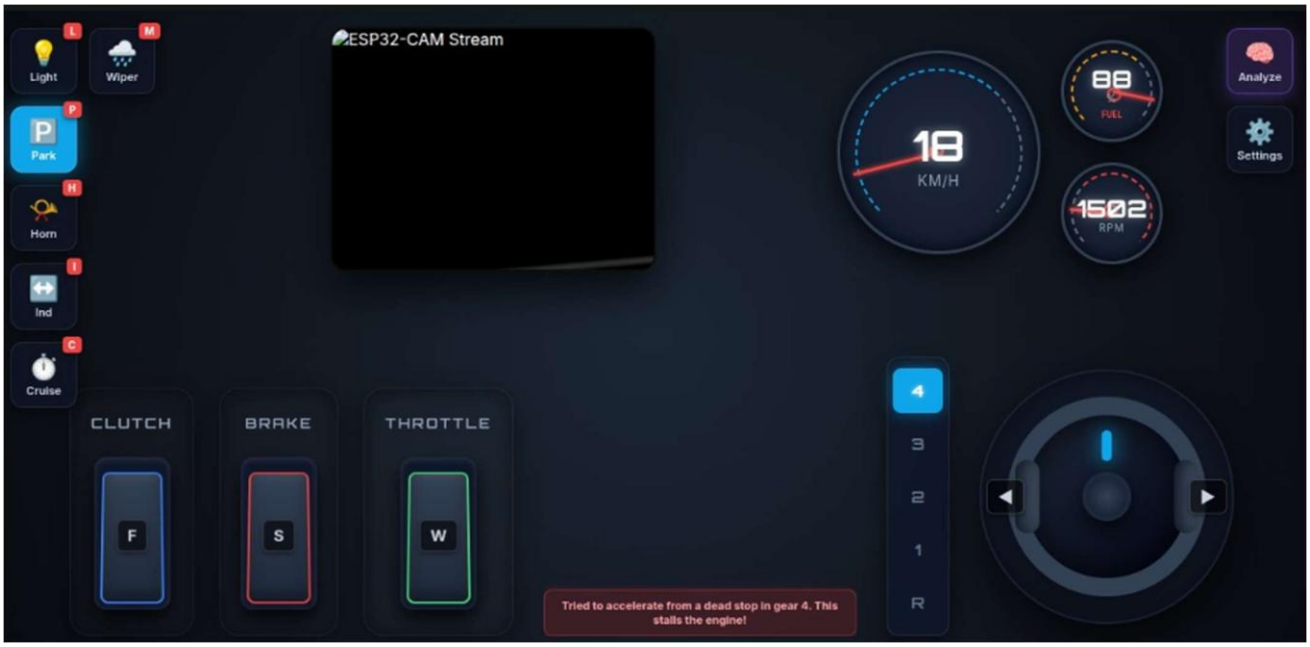
Sudden brake alert

Parking brake

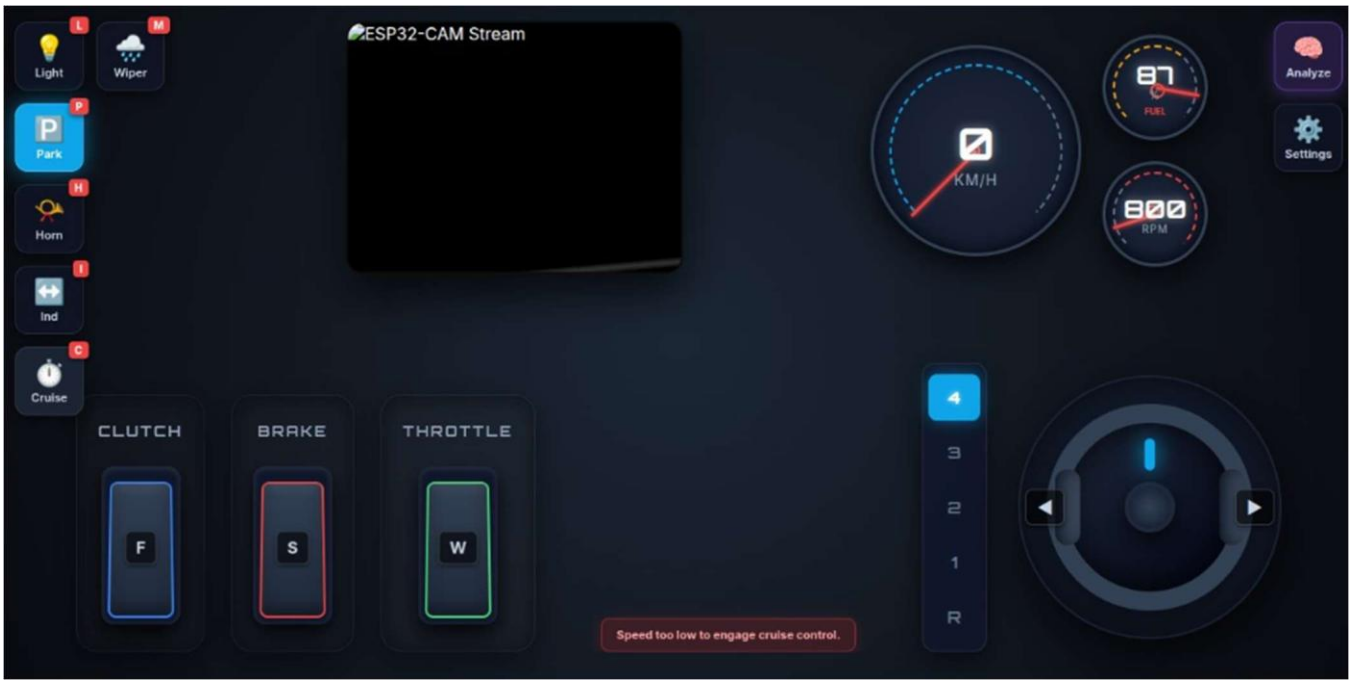
Over speed



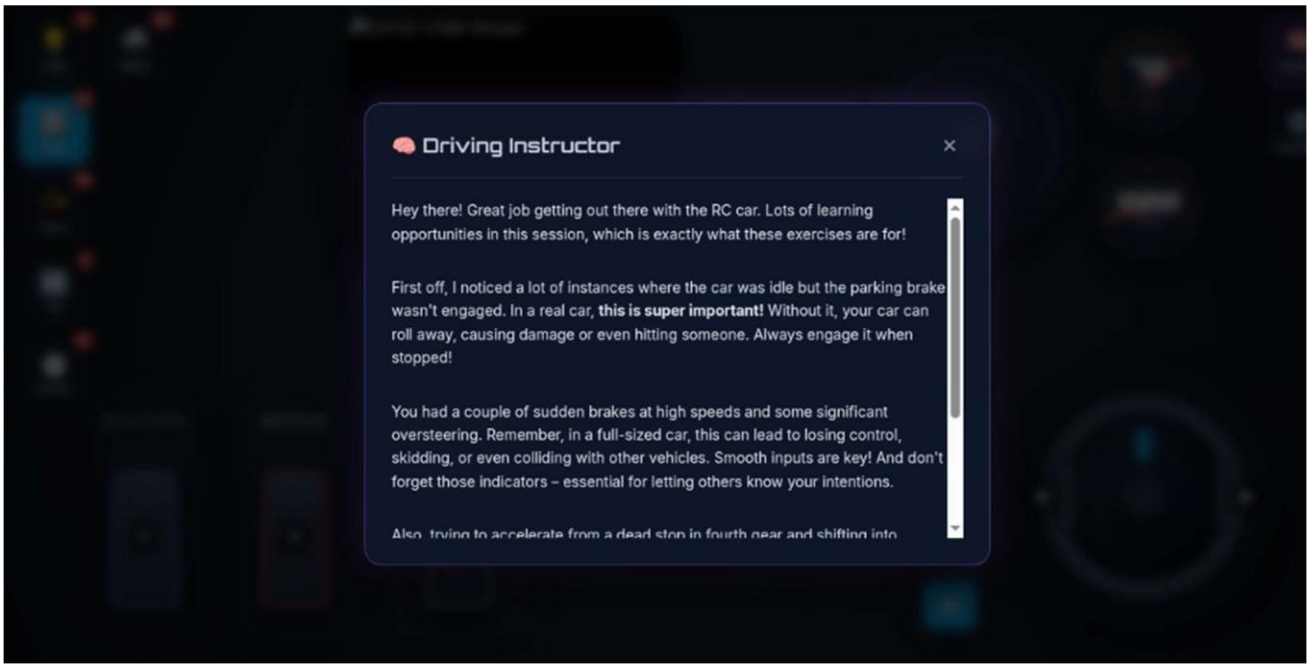
Engine stall



Cruise in low speed



Gemini based analysis for the current drive simulation



CHALLENGES FACED:

Network Flooding & Latency: High-frequency control inputs can easily crash the small ESP8266 web server. Resolving this required implementing intelligent throttling algorithms and state-change checks (isSendingDrive) to ensure the car reflects user inputs faithfully but only transmits necessary updates.

Control Synchronization: Managing concurrent physical/virtual controls—such as ensuring a user must press the clutch key before allowing a gear shift to register, or translating mouse- drag vectors into smooth rotational steering angles—all while guaranteeing identical behavior on both Mobile touches and PC clicks.

Believable Simulation Physics: Tuning the simulated relationship between speed, gears, and RPM so that the dashboard felt realistic. Ensuring that braking halted the RPM accurately and shifting to lower gears while moving at high speeds caused correct speed reductions required intensive timeline tweaking in the core loop

APPLICATIONS:

This system has a wide range of practical and educational applications:

1. Smart Vehicle Prototyping: Acts as a miniature testbed for autonomous and semi-autonomous vehicles. It is also useful for experimenting with driver-assist systems like AEB and AI coaching.

2. Remote Robotics & Teleoperation: Can be used in hazardous environments (inspection, surveillance) where human presence is risky and demonstrates real-time FPV-based remote navigation systems.

3. Educational Platform: Ideal for learning:

- Embedded systems (ESP8266, ESP32-CAM)
- Networking (HTTP communication)
- Frontend development (HTML, CSS, JavaScript)
- AI integration (Gemini API)
- Helps students understand real-world system integration.

4. AI-Assisted Driving Systems: This serves as a prototype for intelligent driver feedback systems in modern vehicles and can be extended into driver training simulators.

5. Human-Machine Interaction Research: It is useful for studying user behavior, control ergonomics, and response times. It also demonstrates how AI can interpret human actions and provide guidance.

CONCLUSION:

This project successfully demonstrates the integration of web technologies, embedded systems, and artificial intelligence into a cohesive remote vehicle control platform. By combining a responsive browser-based interface with real-time hardware communication, the system delivers an immersive and realistic driving experience.

The implementation of features such as FPV streaming, physics-based simulation, AI-driven feedback, and autonomous emergency braking highlights the potential of combining software intelligence with physical systems. Despite challenges like network latency, synchronization of controls, and realistic simulation tuning, effective solutions such as throttling mechanisms and event orchestration ensured smooth performance.

Overall, this project serves as a strong foundation for future advancements in autonomous vehicles, smart safety systems, and AI-assisted driving technologies, demonstrating both practical utility and significant learning outcomes in interdisciplinary engineering.

Video demonstration(Public access):

https://drive.google.com/file/d/1TKA6W2ndQjcjwfD0HwXRoxZghQuYoioB/view?usp=sharing

Code:

https://drive.google.com/drive/folders/1js5YCN2I2YBlH4oy3db1JeFLqARuezp8?usp=sharing

- **index.html,script.js,style.css are used for the web page we use to simulate the drive.**
- **Esp8266_code.ino is the code we use for the esp32-s3 used in our project.**
- **They are all connected using Wi-Fi from a LAN or mobile hotspot.**